

Authentifizierung mit TLS

Andreas Auer, 12317629

Sarah Götz, 12305487

28. Januar 2026

Zusammenfassung

Dieser Report untersucht die Authentifizierung mit TLS und zeigt anhand eines praktischen Man-in-the-Middle-Angriffs, wie fehlerhafte oder unzureichende Zertifikatsvalidierung die Sicherheit moderner Kommunikationsprotokolle untergräbt. Nach einer kompakten Einführung in TLS, den Handshake-Mechanismus und den Unterschied zwischen statischen und ephemeren Schlüsselaustauschverfahren wird ein realer MITM-Angriff mittels ARP-Spoofing umgesetzt. Die Analyse demonstriert, wie sich TLS-Verbindungen trotz Verschlüsselung kompromittieren lassen, wenn Clients manipulierte Zertifikate akzeptieren. Abschließend werden wirksame Gegenmaßnahmen diskutiert.

1 Einleitung

Transport Layer Security (TLS) bildet die Grundlage für sichere Kommunikation im Internet und gewährleistet Vertraulichkeit, Integrität und Authentizität zwischen Client und Server. Während moderne Verfahren wie ECDHE Forward Secrecy und robuste Schlüsselaushandlung ermöglichen, bleibt die tatsächliche Sicherheit wesentlich von der korrekten Validierung digitaler Zertifikate abhängig.

In diesem Report werden zunächst die zentralen Mechanismen von TLS sowie die zugehörigen Handshake-Protokolle erläutert. Anschließend demonstrieren wir in einem isolierten Testnetzwerk, wie ein Angreifer durch ARP-Spoofing und gefälschte Zertifikate eine vollständige TLS-Interception durchführen kann. Die praktische Umsetzung zeigt, dass selbst verschlüsselte Verbindungen verwundbar sind, wenn grundlegende Authentifizierungsmechanismen versagen. Abschließend werden zentrale Schutzmaßnahmen diskutiert, die solche Angriffe effektiv verhindern.

2 Authentifizierung mit TLS

2.1 Einführung in TLS

Das Transport Layer Security (TLS) Protokoll ist ein kryptographisches Protokoll, das die Vertraulichkeit, Integrität und Authentizität der Kommunikation zwischen zwei Endpunkten sicherstellt. Es wird typischerweise über TCP eingesetzt und bildet die

Grundlage für sichere Verbindungen im Internet, beispielsweise bei HTTPS, E-Mail-Verschlüsselung oder VPNs. TLS ist der Nachfolger von SSL (Secure Sockets Layer) und wird kontinuierlich weiterentwickelt, um neuen Sicherheitsanforderungen gerecht zu werden.

Das Hauptziel von TLS ist es, eine sichere Verbindung zwischen Client und Server herzustellen, bei der beide Parteien sicherstellen können, dass keine unbefugten Dritten (Man-in-the-Middle, MITM) den Datenverkehr manipulieren oder abhören können. Dies wird durch eine Kombination aus asymmetrischer und symmetrischer Kryptografie sowie digitalen Zertifikaten erreicht. Während die asymmetrische Kryptografie für den initialen Schlüsselaustausch und die Authentifizierung verantwortlich ist, wird die symmetrische Kryptografie anschließend zur effizienten Verschlüsselung der übertragenen Daten genutzt [Res18].

2.2 Der TLS-Handshake

Der sogenannte *Handshake* ist ein mehrstufiger Prozess, der den sicheren Austausch kryptografischer Parameter zwischen Client und Server ermöglicht. Im Verlauf des Handshakes werden Protokollversionen, unterstützte Cipher Suites und Kompressionsmethoden ausgehandelt. Anschließend erfolgt die Authentifizierung des Servers mittels eines digitalen Zertifikats, das von einer vertrauenswürdigen Zertifizierungsstelle (Certificate Authority, CA) signiert wurde. Optional kann auch der Client authentifiziert werden, beispielsweise in Unternehmensnetzwerken [Res18].

Cipher Suite Eine *Cipher Suite* definiert die Kombination kryptografischer Algorithmen, die für eine bestimmte Verbindung verwendet wird, und legt somit das gesamte Sicherheitsniveau fest. In TLS 1.3 beschreibt eine Cipher Suite ausschließlich die verwendete symmetrische Verschlüsselung sowie die Hashfunktion. Die Cipher Suite `TLS_AES_128_GCM_SHA256` bedeutet beispielsweise, dass die Datenübertragung mittels AES-128 im GCM-Modus als AEAD-Verfahren (Authenticated Encryption with Associated Data) geschützt wird und die Hashfunktion SHA-256 für die schlüsselableitenden Funktionen (HKDF) verwendet wird.

Die Auswahl der Cipher Suite erfolgt weiterhin durch den Server auf Basis der vom Client angebotenen Liste. Schlüsselaustausch und Authentifizierung sind in TLS 1.3 jedoch fest im Protokoll definiert und nicht mehr Teil der Cipher Suite. Dadurch wird die Gefahr kryptographischer Fehlkonfigurationen reduziert und die Sicherheit des Protokolls erhöht [Res18].

2.3 Static Handshake im Detail

Beim *Static Handshake* wird ein fester, im Zertifikat enthaltener öffentlicher Schlüssel des Servers verwendet, um den Sitzungsschlüssel auszutauschen. Der Client verschlüsselt hierbei ein sogenanntes Pre-Master-Secret mit dem öffentlichen Schlüssel des Servers. Der Server entschlüsselt diesen Wert mit seinem privaten Schlüssel und generiert daraus gemeinsam mit dem Client die symmetrischen Sitzungsschlüssel.

Ein Nachteil dieses Ansatzes ist, dass die Sicherheit der Sitzung unmittelbar vom Schutz des privaten Schlüssels abhängt. Sollte dieser kompromittiert werden, kann ein Angreifer auch rückwirkend (z.B. durch zuvor aufgezeichneten Netzwerkverkehr) alle vergangenen Verbindungen entschlüsseln. Dies verletzt das Prinzip der *Forward Secrecy* und gilt daher als veraltet. In Umgebungen mit strikten Ressourcen- oder Kompatibilitätsvorgaben kann es trotzdem erforderlich sein, auf ältere, weniger flexible Handshake-Varianten zurückzugreifen [Res18].

2.4 Ephemeral Diffie-Hellman (ECDHE) Handshake

Um die genannten Schwächen zu beseitigen, verwendet TLS moderne Schlüsselaustauschverfahren wie *Ephemeral Elliptic Curve Diffie-Hellman* (ECDHE). Hierbei handelt es sich um eine Variante des klassischen Diffie-Hellman-Schlüsselaustauschs, die auf elliptischen Kurven basiert und bei jedem Handshake temporäre (ephemere) Schlüsselpaare generiert. Diese werden ausschließlich für die jeweilige Sitzung verwendet und anschließend verworfen.

Der Ablauf lässt sich wie folgt beschreiben: Der Server sendet im Rahmen der *ServerHello*-Nachricht seine temporären elliptischen Kurvenparameter sowie eine digitale Signatur. Der Client generiert ebenfalls ein temporäres Schlüsselpaar, berechnet den gemeinsamen geheimen Wert (Shared Secret) mittels der ECDH-Funktion und sendet seinen öffentlichen Schlüssel an den Server. Beide Seiten berechnen daraus unabhängig denselben Sitzungsschlüssel, ohne dass dieser jemals über das Netzwerk übertragen wird.

Durch den Einsatz ephemerer Schlüssel bietet ECDHE den entscheidenden Vorteil der *Forward Secrecy*: selbst wenn ein Angreifer später Zugriff auf den privaten Schlüssel des Servers erhält, kann er frühere Sitzungen nicht entschlüsseln, da die jeweiligen temporären Schlüssel bereits verworfen wurden. Zudem bietet ECDHE im Vergleich zu klassischen Diffie-Hellman-Verfahren eine höhere Sicherheit bei geringerer Rechenlast, da elliptische Kurven eine kleinere Schlüssellänge bei vergleichbarem Sicherheitsniveau ermöglichen.

ECDHE-basierte Handshakes sind also in sicherheitskritischen Anwendungen zwingend erforderlich, um den Schutz sensibler Daten auch bei späteren Schlüsselkompromittierungen zu gewährleisten. Sie gelten heute als Standard in TLS 1.3 [Res18].

3 Praktische Umsetzung eines Man-in-the-Middle Angriffs

3.1 Grundprinzip des MITM-Angriffs

Ein *Man-in-the-Middle* (MITM)-Angriff bezeichnet eine Attacke, bei der sich ein Angreifer unbemerkt zwischen zwei Kommunikationspartnern schaltet, um Daten abzufangen, zu manipulieren oder weiterzuleiten. In unsicheren Netzwerken, beispielsweise bei fehlender oder unzureichender Authentifizierung, kann der Angreifer den Datenverkehr so umleiten, dass beide Parteien glauben, direkt miteinander zu kommunizieren.

Zu den häufigsten MITM-Angriffen im Zusammenhang mit TLS gehören das Abfangen von HTTPS-Verbindungen durch gefälschte Zertifikate und die Manipulation der

TLS-Handshake-Nachrichten. Dies ist insbesondere dann möglich, wenn der Client unzureichende Zertifikatsvalidierung durchführt oder vom Benutzer angebotene, selbstsignierte Zertifikate akzeptiert [BJR⁺14].

3.2 ARP-Spoofing als Grundlage des MITM

Eine verbreitete Technik, um einen MITM-Angriff auf Netzwerkebene zu initiieren, ist *Address Resolution Protocol (ARP) Spoofing* (auch ARP Poisoning). Dabei sendet der Angreifer gefälschte ARP-Antworten an die Opfergeräte, sodass seine eigene MAC-Adresse fälschlich mit der IP-Adresse eines legitimen Kommunikationspartners assoziiert wird. Der Verkehr zwischen Client und Server kann dadurch über den Angreifer umgeleitet werden [Cis]. In dieser Position kann der Angreifer den TLS-Verkehr zwar umleiten und beeinflussen, ein erfolgreiches Mitlesen oder Manipulieren setzt jedoch voraus, dass die Authentifizierung im TLS-Handshake scheitert. Ist die Zertifikatsvalidierung fehlerhaft, kann der Angreifer ein gefälschtes Zertifikat präsentieren und damit einen MITM auf TLS-Ebene ermöglichen, wodurch Vertraulichkeit und Integrität der Kommunikation nicht mehr gewährleistet sind [BJR⁺14].

3.3 Aufbau

Für die praktische Demonstration eines MITM-Angriffs wird ein isoliertes Testnetzwerk aufgebaut, bestehend aus drei virtuellen Maschinen (Kali Linux) mittels Parallels Desktop: einem *Client*, einem *Server* und einem *MITM*-System. Das Netzwerk ist vollständig vom Internet getrennt und lediglich untereinander sowie mit dem Host (MacBook) verbunden. Der Server stellt über Apache 2 eine kleine Webanwendung (HTTPS) bereit. Der Client greift über Chrome darauf zu. Das MITM-System führt den Angriff mittels `bettercap` und `mitmproxy` durch [Evi, Cor].

3.4 Übersicht des Angriffs

(1) ARP-Spoofing Durch ARP Spoofing werden die beiden Opfergeräte (*Alice* und *Bob*) dazu gebracht, die MAC-Adresse des Angreifers (*Mallory*) fälschlicherweise der IP-Adresse des jeweils anderen Systems zuzuordnen. So wird der gesamte Datenverkehr zwischen *Alice* und *Bob* über *Mallory* geleitet (siehe Abb. 1).

(2) TLS-Interception Nachdem der Datenverkehr erfolgreich umgeleitet ist, versucht *Mallory*, den TLS-Handshake des Clients abzufangen. Damit dies gelingt, muss *Mallory* ein vom Client akzeptiertes Zertifikat für die Ziel-Domain präsentieren – sei es über eine kompromittierte bzw. installierte Root-CA, ein manipuliertes Zertifikats-Setup oder ein vom Benutzer bestätigtes selbstsigniertes Zertifikat.

Akzeptiert der Client das Zertifikat, entsteht eine sogenannte TLS-Bridge: *Alice* baut eine TLS-Verbindung zu *Mallory* auf, während *Mallory* zeitgleich eine separate TLS-Verbindung zum echten Server *Bob* errichtet (siehe Abb. 2). *Mallory* kennt dadurch

sämtliche Sitzungsschlüssel und kann Inhalte entschlüsseln, einsehen, manipulieren und erneut verschlüsselt weiterleiten.

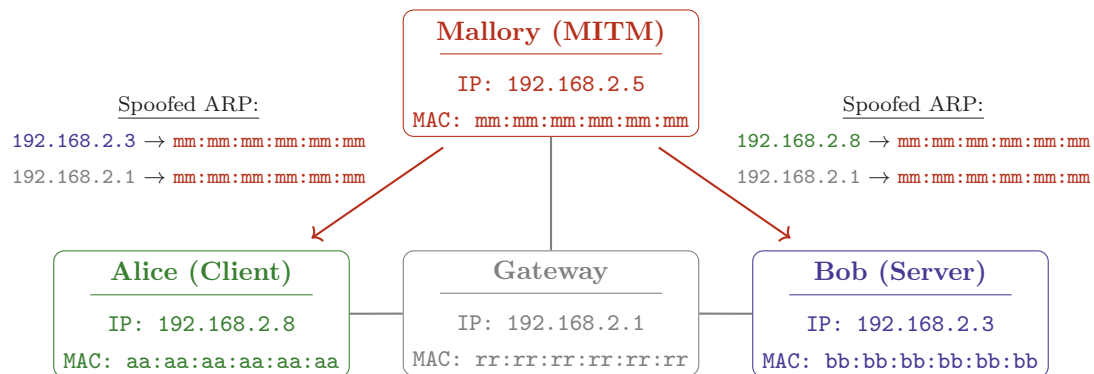


Abbildung 1: ARP-Spoofing

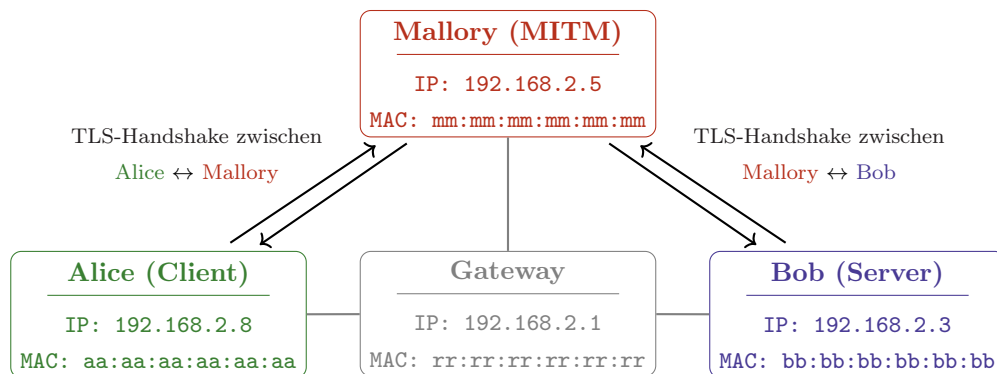


Abbildung 2: TLS-Interception

3.5 Umsetzung

3.5.1 Netzkumleitung durch ARP-Spoofing

Mallory startet `bettercap` und aktiviert ARP-Spoofing mithilfe eines Caplets, das zuerst Geräte im Netzwerk scannt und anschließend das Spoofing einschaltet. Danach verknüpfen **Alice** und **Bob** alle relevanten IP-Adressen in ihren ARP-Tabellen mit der MAC-Adresse von **Mallory**. Damit wird der gesamte IPv4-Verkehr transparent über **Mallory** geleitet.

```
# CAPLET: arp_spoofing.cap
net.probe on; sleep 3; net.probe off # Netzwerkscan, um Geräte zu finden
set arp.spoof.internal true # alle internen IP-Adressen spoofen
arp.spoof on # spoofing starten
net.show
```

```
$ sudo bettercap -iface eth0 -caplet /Documents/arp_spoofing.cap
```

IP	MAC	Name	Vendor	Sent	Recv	Seen
192.168.2.5	00:1c:42:57:2d:d9	eth0	...	0 B	0 B	23:59:25
192.168.2.1	00:1c:42:00:00:18	gateway	...	2.2 kB	2 kB	23:59:25
192.168.2.3	00:1c:42:4d:6a:1e	server	...	1.6 kB	2 kB	00:00:42
192.168.2.8	00:1c:42:93:3a:b8	client	...	3.4 kB	3 kB	00:00:43

Nach Aktivierung des ARP-Spoofings verknüpfen sowohl **Alice** als auch **Bob** sämtliche relevanten IP-Adressen in ihren ARP-Tabellen mit der MAC-Adresse von **Mallory**. Dadurch läuft jede unverschlüsselte Kommunikation sofort für **Mallory** im Klartext ein. Die ARP-Tabelle von **Alice** zeigt dies exemplarisch; **Bob** weist eine identische Struktur auf.

```
$ arp -n
```

Address	HWtype	HWaddress	Flags	Iface
192.168.2.3	ether	00:1c:42:57:2d:d9	C	eth0
192.168.2.5	ether	00:1c:42:57:2d:d9	C	eth0
192.168.2.1	ether	00:1c:42:57:2d:d9	C	eth0

Ab diesem Punkt läuft der gesamte Datenverkehr über **Mallory**, was ein **traceroute** von **Alice** bestätigt: die Anfrage von **Alice** an **testserver.local** geht zunächst an **Mallory** und erst dann an **Bob**.

```
$ traceroute testserver.local
traceroute to testserver.local (192.168.2.3), 30 hops max, 60 byte packets
1  kali-linux-2025.2-arm64 (192.168.2.5)  0.461 ms  0.417 ms  0.404 ms
2  testserver.local (192.168.2.3)  0.811 ms  0.805 ms  0.754 ms
```

3.5.2 Abfangen verschlüsselter Daten

Um auch TLS-geschützten Verkehr einsehen zu können, muss **Mallory** als TLS-Interception-Proxy agieren. Dafür wird **mitmproxy** im transparenten Modus betrieben [Cor]. Eingehende HTTP(S)-Verbindungen werden per Portweiterleitung dorthin umgeleitet (von 443 bzw. 80 zu 8084), wodurch eine TLS-Bridge entsteht, über die Inhalte im Klartext sichtbar werden.

```
$ sudo iptables -t nat -A PREROUTING -p tcp --dport 80 \
-j REDIRECT --to-ports 8084
$ sudo iptables -t nat -A PREROUTING -p tcp --dport 443 \
-j REDIRECT --to-ports 8084

$ sudo SSL_CERT_FILE=/path/to/certs.crt \
mitmproxy --mode transparent --listen-port 8084
```

3.5.3 Zertifikatsbasierte Authentifizierung in TLS

Ein MITM-Angriff auf eine TLS-Verbindung gelingt nur, wenn der Client das Zertifikat des Angreifers akzeptiert. Dies kann passieren, wenn Warnungen ignoriert, manipulierte Root-CAs installiert oder Zertifikate unzureichend geprüft werden. Da TLS seine Authentizität allein über Zertifikatsketten sicherstellt, ist korrekte Validierung entscheidend. Damit ein Client einem TLS-Server vertraut, muss dessen Zertifikat bis zu einer Root-CA zurückführbar sein, der der Client vertraut. Im Folgenden richten wir eine eigene CA ein, die für die legitime Serverkommunikation genutzt wird.

Einrichtung und Installation einer eigenen Certificate Authority (CA) Zur Demonstration, wird zunächst eine eigene Root-CA erzeugt. Mit dieser Root-CA signieren wir anschließend das Serverzertifikat, das später vom Client akzeptiert werden soll. Dazu erstellen wir zunächst das Root-Zertifikat samt privatem Schlüssel und leiten daraus im nächsten Schritt das signierte Serverzertifikat ab (nach üblichem Vorgehen).

Wird nun das Root-Zertifikat der CA bei **Alice** installiert, vertraut diese automatisch allen von ihr signierten Serverzertifikaten. Dadurch lässt sich zunächst eine reguläre, korrekte TLS-Verbindung demonstrieren – als Grundlage, um anschließend den MITM-Angriff und die missbrauchte Vertrauenskette zu zeigen.

Zertifikate im MITM-Szenario Für den MITM-Angriff erzeugt **Mallory** eine eigene Zertifikatskette (automatisch durch `mitmproxy` erstellt). Sobald **Alice** eine HTTPS-Verbindung zu `testserver.local` aufbaut, präsentiert **Mallory** ein neu signiertes Zertifikat für dieselbe Domain. Akzeptiert **Alice** dieses Zertifikat, entstehen zwei getrennte TLS-Verbindungen:

$$\text{Alice} \xleftrightarrow{\text{TLS}} \text{Mallory} \xleftrightarrow{\text{TLS}} \text{Bob}$$

Da **Mallory** beide Sitzungsschlüssel kennt, kann der gesamte Traffic eingesehen, verändert und wieder verschlüsselt weitergegeben werden.

4 Gegenmaßnahmen und Wiederherstellung der Sicherheit

Zur Wiederherstellung der Netzwerksicherheit und zum Schutz vor MITM-Angriffen sind mehrere Maßnahmen wirksam:

- Korrekte Zertifikatsvalidierung: TLS-Implementierungen müssen X.509-Zertifikate strikt prüfen und selbstsignierte oder abgelaufene Zertifikate ablehnen [BJR⁺14].
- ARP-Schutzmechanismen: Einsatz von statischen ARP-Tabellen oder von Mechanismen, die ARP-Spoofing erkennen und blockieren.
- Verschärfte Benutzeraufklärung: Anwender müssen auf Warnungen zu ungültigen Zertifikaten reagieren und diese nicht ignorieren.

Ein zentraler Aspekt der Gegenmaßnahmen liegt dabei in der korrekten Implementierung und Durchsetzung der Zertifikatsprüfung auf Client-Seite. Selbst weit verbreitete SSL/TLS-Implementierungen treffen in der Praxis häufig inkonsistente oder fehlerhafte Entscheidungen bei der Validierung von Zertifikaten [BJR⁺14]. Solche Abweichungen von der Spezifikation können es Angreifern ermöglichen, trotz kryptographisch gesicherter Verbindungen gefälschte Zertifikate einzusetzen und somit eine TLS-Interception erfolgreich durchzuführen. Besonders problematisch sind dabei Randfälle in der Zertifikatskette oder unzureichend behandelte Fehlersituationen, in denen Verbindungen nicht konsequent abgebrochen werden.

Diese Maßnahmen gewährleisten, dass auch bei kompromittierten Netzwerken die Authentizität und Integrität der Kommunikation wiederhergestellt werden kann. In modernen Systemen stellt insbesondere TLS 1.3 durch die verpflichtende Verwendung von ECDHE und eine strengere Handhabung der Zertifikatsvalidierung einen hohen Schutz gegen MITM-Angriffe sicher [Res18].

5 Fazit

Die Analyse zeigt, dass die Sicherheit von TLS maßgeblich von einer korrekten Authentifizierung und der Integrität der zugrunde liegenden Netzwerkumgebung abhängt. Der durchgeführte MITM-Angriff verdeutlicht, dass selbst starke kryptografische Verfahren wirkungslos werden, wenn Zertifikate nicht strikt geprüft oder grundlegende Netzwerkprotokolle wie ARP manipulierbar sind. Gleichzeitig bestätigt die Untersuchung, dass moderne TLS-Versionen – insbesondere TLS 1.3 mit verpflichtendem ECDHE – einen hohen Schutz bieten, sofern Implementierung und Validierung korrekt erfolgen. Effektive Gegenmaßnahmen wie robuste Zertifikatsprüfung, ARP-Schutzmechanismen und Benutzeraufklärung sind daher entscheidend, um die Vertraulichkeit und Integrität sicherer Kommunikation nachhaltig zu gewährleisten.

Literatur

- [BJR⁺14] C. Brubaker, S. Jana, B. Ray, S. Khurshid, and V. Shmatikov. Using frankencerts for automated adversarial testing of certificate validation in ssl/tls implementations. *IEEE Symposium on Security and Privacy*, 2014.
- [Cis] Cisco Systems. Configuring dynamic arp inspection — arp spoofing / arp cache poisoning. Cisco Documentation. Accessed December 2025.
- [Cor] Cortesi, Aldo and Hils, Maximilian and contributors. mitmproxy. <https://mitmproxy.org/>. Accessed December 2025.
- [Evi] Evilsocket (Simone Margaritelli). bettercap. <https://www.bettercap.org/>. Accessed December 2025.
- [Res18] Eric Rescorla. The transport layer security (tls) protocol version 1.3. RFC 8446, Internet Engineering Task Force (IETF), 2018. RFC 8446.